

Task 5: Scaling the Audio App to 5,000 Workers

Prepared by Sanchay Roy

Current Situation

The audio app is a single Streamlit process. Every submission is handled synchronously in the same request: the audio bytes are read, duration, sample rate, bitrate, loudness, noise floor, and a quality score are computed with numpy right there in the request, the file is written to a local folder on disk with a UUID filename, and a row is written to a local SQLite database file. Identity matching against the merged people table is a simple phone-plus-name lookup. There is no queue, no object storage, no authentication, and nothing that stops the same person from submitting more than once.

What Breaks First

SQLite allows only one writer at a time. As soon as more than a handful of people submit audio in the same few seconds, some of those writes will fail or wait, and 5,000 workers over a single weekend will produce exactly that kind of concurrent burst, especially in the evenings when people are likely to sit down and record.

The audio analysis itself runs inside the request, so a user is stuck waiting on the numpy computation before they get a response, and depending on how the app is hosted, one slow analysis can hold up others behind it. Local disk storage is also a real risk: on typical free hosting for a Streamlit app, the filesystem is often not guaranteed to persist across restarts, so uploaded audio could be lost without warning.

Changes Before Launch

- Move audio files to object storage (S3 or similar) right after upload instead of a local folder, so files survive restarts and are not tied to one server's disk.
- Move the metadata database from SQLite to Postgres. The schema is already relational, so this is mostly a connection change, and it removes the single-writer bottleneck.
- Decouple audio analysis from the upload request: accept and store the file quickly, then run the actual duration and loudness computation as a background job, and update the row once it finishes, so a user is not stuck waiting on numpy.

Handling Failures and Duplicates

Right now nothing stops one worker from submitting audio five times, each one creating a new row, since matching only checks whether the person exists, not whether they already submitted. Before launch I would add a simple check per phone number, either limit to one active submission or make repeat submissions clearly visible instead of piling up silently. A failed or interrupted upload today just raises an error and nothing is written, which is a safe default, but there is no retry, so a flaky connection during a weekend spike would mean some workers give up rather than succeed on a second try. A basic client-side retry would help.

Storage and Cost

Even a generous 2 to 3 MB WAV file per submission across 5,000 workers is only 10 to 15 GB, which is cheap on object storage. The bigger cost is compute time from running audio analysis synchronously on every request, that is the part that needs to move to a background job before the storage question matters much.

Final Recommendation

The current schema and the audio analysis logic itself do not need to change, they already extract the right fields correctly. What needs to change before 5,000 workers is where the file lives, where the database lives, and when the analysis actually runs: object storage instead of local disk, Postgres instead of SQLite, and a background job instead of doing the computation inline during the upload request.